

ProofSettle

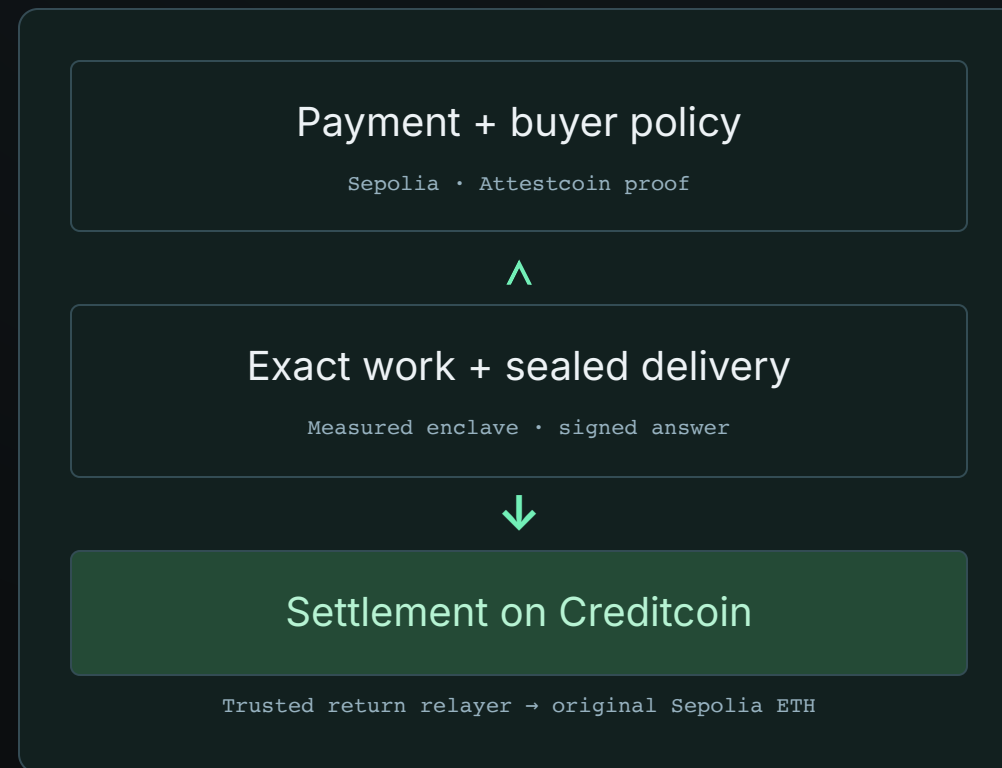
Pay for the exact work you requested.

Two independent proofs authorize settlement.
The request and answer stay sealed.

Attestcoin

Creditcoin

Confidential Space



A lender needs a decision. The provider must not get the record.

Imagine a small lender scoring an applicant with six months of mobile-money history and no conventional credit file.

The lender can compute the features. A specialist supplies the model and compute. Neither a payment receipt nor a provider's promise establishes that the requested private computation was delivered.

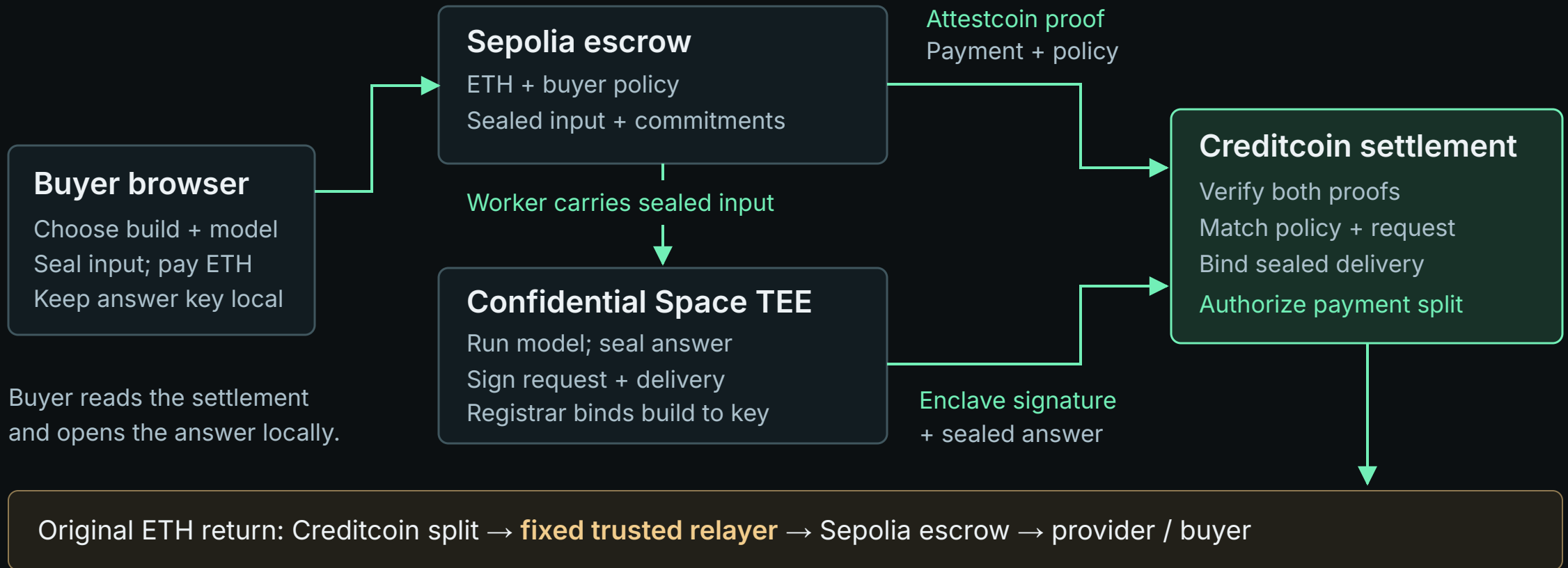
The buyer controls the acceptable build, model and input in the payment itself.

ILLUSTRATIVE SYNTHETIC APPLICANT

Transaction history	6 months
Regular inflows	14 / month
Prior loans	None

The demo uses a small deterministic logistic model. It demonstrates verified service delivery, not validated credit underwriting.

Two proofs meet at Creditcoin



The worker submits both proofs in one settlement transaction. A computed decline still pays for successful service; the applicant's decision stays sealed.

The proven payment carries executable policy.

JobCreated

- | buyer + provider + amount
- | required enclave measurement
- | model hash + salted input hash
- | encrypted envelope hash
- └ settlement deadline

Prove inclusion + continuity

Native verifier at 0x...0FD2; false returns fail closed.

Decode the successful receipt

EvmV1Decoder; pin the escrow emitter and event signature.

Enforce the buyer's policy

Read the required build and request from that proven event.

Consume once

calculateTxIndex produces the replay key.

RawProofBuilder constructs proofs locally. ChainInfo at 0x...0FD3 reports the attested height. The hosted builder is a fallback, not an authority.

A genuine enclave signature is not enough.

A

Bind the request

keccak256(model hash, input hash, encrypted-envelope hash). Swapping the return key changes the envelope.

B

Bind the delivery

The signature commits to the exact payload appended to settlement. Dropping or replacing it fails.

C

Bind the domain

Job ID, chain ID, settlement address, result, service outcome and split are signed together.

The contract compares the signed request to the proven payment, and the signed delivery to the actual transaction bytes.

RequestMismatch and DeliveryMismatch are tested with real signatures. Reverted checks roll back proof consumption.

Follow the original payment all the way through.

01 · Paid on Sepolia

0x268bd557f6674c252a61d0f04360e864ff664750bb7f3025ef95cc596231f964

02 · Settled on Creditcoin

0x7a0682124d901571bab78ebf0de8e76736cf9e89ab15be436eec9c6919211afa

03 · ETH withdrawn on Sepolia

0x8d1610259edfb84015e5130a6a4f4634aabee8514922f27a2a347101954df072

Open the synthetic answer.

proofsettle.pages.dev/desk.html?demo=1

Real testnet transactions; deliberately public synthetic data and a disposable answer key. The homepage reads both chains live. A new buyer's key remains in their browser.

The wrong build gets a mined refusal.

The buyer pays successfully but names a build the serving enclave does not match.

Attestcoin proves the payment. The enclave supplies a genuine signature. Creditcoin still refuses it.

EnclaveNotAccepted

No settlement is recorded. The payment remains in escrow and is reclaimable after the source timeout.

MINED CREDITCOIN TRANSACTION • STATUS 0

0x9e4cd30ef97ac952f999098d39bc6c54f89dc1fa
3c3bd5596a11e60caee38777

The live homepage reproduces the named error with a historical call. A reverted transaction is evidence of enforcement, not just a red screen.

[Inspect the fifth live check →](#)

Test the guarantee. Then remove it.

65

contract tests passing

15

security mutations caught

10

enclave integration tests

Adversarial cases

Substitute input, model or return key. Drop the delivered answer. Replay the source proof. Ignore a false oracle return. Upgrade a verdict. Refund an already released payment.

Beyond unit tests

Real enclave process, cryptographic round trip, browser wallet workflow, bad-attestation cases, deployment decisions, live chain readbacks and published source verification.

```
npm run judge:verify
```

Local tests use explicit fixtures where a native precompile or wallet is unavailable. Live readbacks are separate and labelled.

Precise claims are part of the product.

1

Public-chain privacy

Addresses, amounts, commitments and ciphertext are public.
Plaintext inputs and accepted answers are encrypted.

2

Hardware enrollment

Google and the hardware root are trusted. A registrar binds the verified image to its key; the evidence is public.

3

Original ETH payout

A fixed trusted return relayer acts on Creditcoin state. This is not production Attestcoin writability.

4

Testnet limits

Public testnet prototype; synthetic model. Independent security review and underwriting validation precede production use.

The destination receipt token is non-transferable accounting history. It is not a second redeemable currency.

Start with one lender and one model provider.

30 DAYS

Validate the workflow

Interview lender operations and model providers. Secure a design partner; agree a synthetic evaluation set and delivery requirements.

60 DAYS

Run a bounded pilot

Integrate one provider. Measure completion rate, proof latency, total cost and buyer recovery. Review the contracts and enrollment process independently.

90 DAYS

Earn expansion

Proceed only if the pilot demonstrates value. Add model/version lifecycle and recovery before supporting larger workloads or real borrower data.

Commercial hypothesis: a per-settlement integration/service fee. Pricing and willingness to pay must be validated with the pilot.

The next milestone is customer discovery and a design partner. The small model isolates settlement correctness; GPU-scale execution is future work.

A focused settlement primitive for private compute.

Approach	What it establishes	What ProofSettle adds
Compute-payment marketplace	Purchase, credits and metering	Buyer-selected measured execution with sealed input and output
TEE inference endpoint	A measured workload and signed answer	A proven cross-chain payment bound to that exact request and delivery
Cross-chain payment proof	An event really happened	Policy from the event enforced against an independent proof system

The differentiation is the composition: payment proof × measured work × exact request × delivered ciphertext.

These are architectural categories, not a claim that no other entry can implement them.

Inspect the work. Choose your depth.

Robert Moore · solo builder

Contracts, enclave, proof worker, browser workflow and testnet deployment. The implementation and evidence are public.

Seeking a lender/model-provider design partner and an independent security review for the next stage.

MIT open source

Sepolia + CC3

Verify a real run →

proofsettle.pages.dev

Read the code →

github.com/iamrobertmoore/proofsettle

```
npm run judge:verify
```